

# Fighting Bit rot

With your mind

Viktor Klang

# Bit rot

The term "bit rot" is often used to refer to dormant code rot, i.e. **the fact that dormant (unused or little-used) code gradually decays in correctness** as a result of interface changes in active code that is called from the dormant code.

- [http://en.wikipedia.org/wiki/Bit\\_rot](http://en.wikipedia.org/wiki/Bit_rot)



... or how to create  
maintainable software

I happen to hold a hard-won minority opinion about code bases. In particular I believe, quite staunchly I might add, that **the worst thing that can happen to a code base is size.**

– Steve Yegge

# Outline

# Outline

- Example: Akka

# Outline

- Example: Akka
- Code & Maintenance

# Outline

- Example: Akka
- Code & Maintenance
- How our minds fight bit rot

# Outline

- Example: Akka
- Code & Maintenance
- How our minds fight bit rot
- Tools

# Outline

- Example: Akka
- Code & Maintenance
- How our minds fight bit rot
- Tools
- Pro tips

# Outline

- Example: Akka
- Code & Maintenance
- How our minds fight bit rot
- Tools
- Pro tips
- Summary



Build powerful,  
concurrent & distributed  
applications more easily.



# Akka stats

# Akka stats

● Number of modules: **19**

# Akka stats

- Number of modules: **19**
- First Commit: Feb 2009

# Akka stats

- Number of modules: **19**
- First Commit: Feb 2009
- Total Commits: **13,450**

# Akka stats

- Number of modules: **19**
- First Commit: Feb 2009
- Total Commits: **13,450**
- Total contributors: 135

# Akka stats

- Number of modules: **19**
- First Commit: Feb 2009
- Total Commits: **13,450**
- Total contributors: 135
- Est. effort: 36 years (COCOMO)

# Akka stats

- Number of modules: **19**
- First Commit: Feb 2009
- Total Commits: **13,450**
- Total contributors: 135
- Est. effort: 36 years (COCOMO)
- Mostly written in **Scala**

<http://www.ohloh.net/p/akka>

# Akka stats

# Akka stats

● Total lines: 203,515

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**
- Percent Code Lines 70.2%

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**
- Percent Code Lines 70.2%
- Number of Languages: 11

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**
- Percent Code Lines 70.2%
- Number of Languages: 11
- Total Comment Lines: 36,416

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**
- Percent Code Lines 70.2%
- Number of Languages: 11
- Total Comment Lines: 36,416
- Percent Comment Lines: 17.9%

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**
- Percent Code Lines 70.2%
- Number of Languages: 11
- Total Comment Lines: 36,416
- Percent Comment Lines: 17.9%
- Total Blank Lines: 24,205

# Akka stats

- Total lines: 203,515
- Code lines: **142,894**
- Percent Code Lines 70.2%
- Number of Languages: 11
- Total Comment Lines: 36,416
- Percent Comment Lines: 17.9%
- Total Blank Lines: 24,205
- Percent Blank Lines: 11.9%

# #1 top Github pull award

with



!

C

=

>

#1

AWARD

gh TDP

PULLS

If debugging is the  
process of removing  
bugs, then  
programming must  
be the process of  
putting them in.

– Edsger Dijkstra

# Maintenance

# Maintenance

- Adaptive – evolve with new surroundings

# Maintenance

- Adaptive – evolve with new surroundings
- Perfective – evolve to meet new needs

# Maintenance

- Adaptive – evolve with new surroundings
- Perfective – evolve to meet new needs
- Corrective – diagnose and fix defects

# Maintenance

- Adaptive – evolve with new surroundings
- Perfective – evolve to meet new needs
- Corrective – diagnose and fix defects
- Preventive – refactor etc

# Maintenance

- Adaptive – evolve with new surroundings
- Perfective – evolve to meet new needs

[...] a good programmer can reasonably maintain about **20,000** lines of code.

[...] language doesn't matter. It's still 20,000 lines.

If your language requires fewer lines to express the same ideas, you can spend more time on stuff that otherwise would go beyond those 20,000 lines.

– Guido van Rossum  
Creator of the Python Programming Language

So ... how do you fight bit  
rot with your **mind**?



# Zero Known Defects Policy

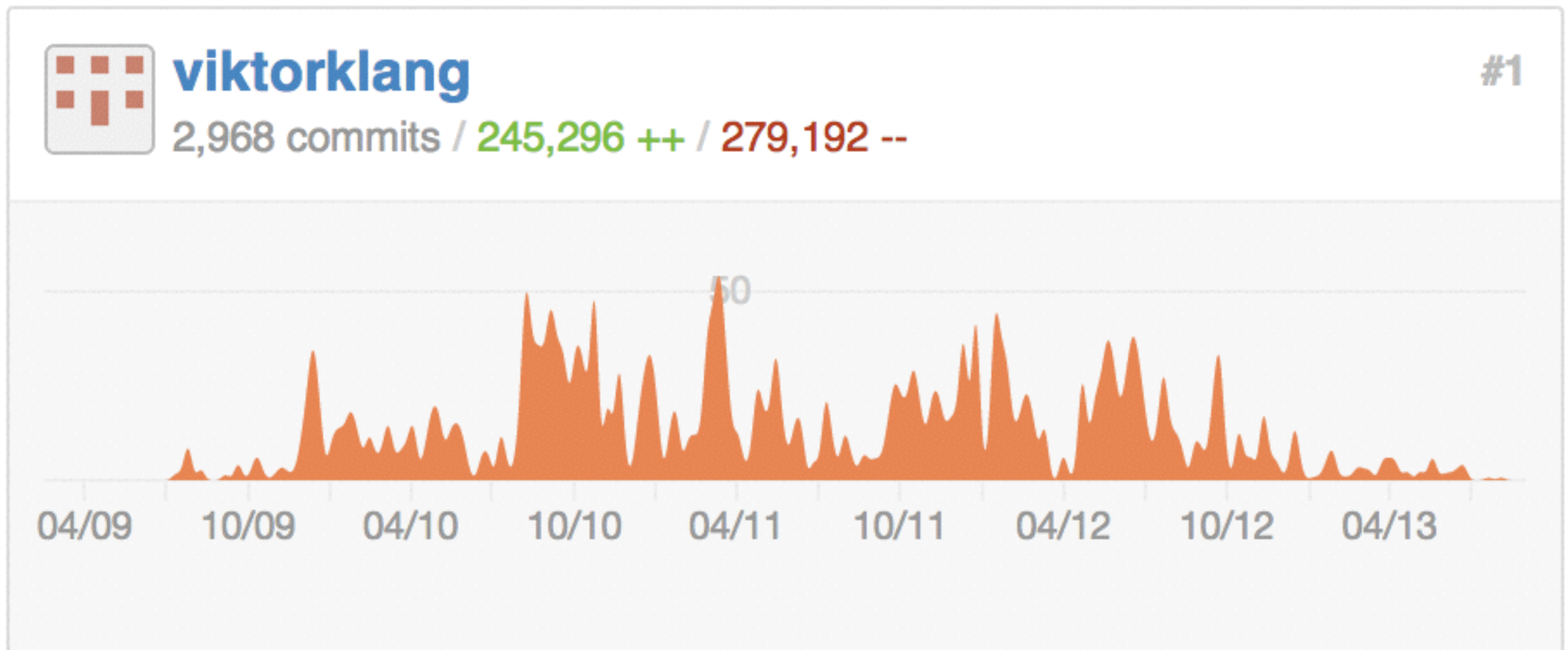
My point today is that, **if we wish to count lines of code, we should not regard them as "lines produced" but as "lines spent":**

the current conventional wisdom is so foolish as to book that count on the wrong side of the ledger.

**– Edsger Dijkstra**

# Lines of Code **Spent**

# Less is more!



# (My) Test philosophy

# (My) Test philosophy

● Feature – **must have**

# (My) Test philosophy

- Feature – **must have**
- Integration – **should have**

# (My) Test philosophy

- Feature – **must** have
- Integration – **should** have
- Unit – **may** have

# Laws

# Laws

☒ DRY

# Laws

- ☒ DRY
- ☒ Boy Scout Rule

# Laws

- DRY
- Boy Scout Rule
- Test

# Laws

- DRY
- Boy Scout Rule
- Test
- Document

# Everyone code reviews

# Everyone code reviews

☒ Collective Ownership

# Everyone code reviews

- Collective Ownership
- Harmonization

# Everyone code reviews

- Collective Ownership
- Harmonization
- Education

# Everyone code reviews

- Collective Ownership
- Harmonization
- Education
- Documentation

# Everyone code reviews

- Collective Ownership
- Harmonization
- Education
- Documentation
- Test Coverage

# Documentation

# Documentation

- Reference documentation

# Documentation

- ⦿ Reference documentation
- ⦿ API documentation

# Documentation

- ⦿ Reference documentation
- ⦿ API documentation
- ⦿ Put “**Whys**” in the code

Solve everything  
at least twice

@author

git blame / praise

# Rules

# Rules

- Code formatting

# Rules

- Code formatting
- Tabs vs spaces

# Rules

- Code formatting
- Tabs vs spaces
- Features

# Rules

- Code formatting
- Tabs vs spaces
- Features
- Commits

# Rules

- ⦿ Code formatting
- ⦿ Tabs vs spaces
- ⦿ Features
- ⦿ Commits
  - ⦿ Description

# Rules

- ⦿ Code formatting
- ⦿ Tabs vs spaces
- ⦿ Features
- ⦿ Commits
  - ⦿ Description
  - ⦿ Message

# Auto-enforce

# Auto-enforce

- ☒ Reformat on save / compile

# Auto-enforce

- Reformat on save / compile
- Post-commit hooks

# Features

- What features do we use when and why?

# Common Dev Tools

# Common Dev Tools

● Issue tracker

# Common Dev Tools

- Issue tracker
- Code reviewer

# Common Dev Tools

- Issue tracker
- Code reviewer
- Continuous Integration

# Common Dev Tools

- ⦿ Issue tracker
- ⦿ Code reviewer
- ⦿ Continuous Integration
- ⦿ Build tool

# Common Dev Tools

- Issue tracker
- Code reviewer
- Continuous Integration
- Build tool
- Source Control

# Common Dev Tools

- Issue tracker
- Code reviewer
- Continuous Integration
- Build tool
- Source Control
- Documentation Generator

# Issue tracking

# Issue tracking

- Absolute **priority** ordering

# Issue tracking

- Absolute **priority** ordering
- Break down into **half-day** size

# Issue tracking

- Absolute **priority** ordering
- Break down into **half-day** size
- Follow **motivation** when choosing

# Source Related

# Source Related

- Code review tool

# Source Related

- Code review tool
- Use the one that works best

# Source Related

- Code review tool
  - Use the one that works best
- Continuous Integration tool

# Source Related

- Code review tool
  - Use the one that works best
- Continuous Integration tool
  - Use the one that works best

# Source Related

- ⦿ Code review tool
  - ⦿ Use the one that works best
- ⦿ Continuous Integration tool
  - ⦿ Use the one that works best
- ⦿ Build tool

# Source Related

- ⦿ Code review tool
  - ⦿ Use the one that works best
- ⦿ Continuous Integration tool
  - ⦿ Use the one that works best
- ⦿ Build tool
  - ⦿ Use the one that works best

# Source Related

- Code review tool
  - Use the one that works best
- Continuous Integration tool
  - Use the one that works best
- Build tool
  - Use the one that works best
- Source Control

# Source Related

- Code review tool
  - Use the one that works best
- Continuous Integration tool
  - Use the one that works best
- Build tool
  - Use the one that works best
- Source Control
  - Use Git and be done with it!

# Continuous Integration

# Continuous Integration

- Run CI on Pull Requests **before** merge

# Continuous Integration

- Run CI on Pull Requests **before** merge
- Run CI continuously on **main** branches

# Documentation Generator

# Documentation Generator

- All code samples should be tests that are compiled and run and automatically lifted in

# Documentation Generator

- All code samples should be tests that are compiled and run and automagically lifted in
- Documentation generation should be a part of CI so if it fails, it fails the build

# Documentation Generator

- All code samples should be tests that are compiled and run and automagically lifted in
- Documentation generation should be a part of CI so if it fails, it fails the build
- Generate at least HTML + PDF

# Individual Tools

# Editors/IDEs

- Build tool will enforce code style so  
use the editor you are  
most productive in!

# HW Interfaces

# HW Interfaces

- Don't change keyboards/layouts to write faster

# HW Interfaces

- ⦿ Don't change keyboards/layouts to write faster
- ⦿ Do it to **avoid strain** on your fingers and wrists

# HW Interfaces

- Don't change keyboards/layouts to write faster
- Do it to **avoid strain** on your fingers and wrists

**10: Pain leads to shortcuts**

# HW Interfaces

- Don't change keyboards/layouts to write faster
- Do it to **avoid strain** on your fingers and wrists

**10: Pain leads to shortcuts**

**20: Shortcuts lead to bugs**

# HW Interfaces

- Don't change keyboards/layouts to write faster
- Do it to **avoid strain** on your fingers and wrists

**10: Pain leads to shortcuts**

**20: Shortcuts lead to bugs**

**30: Bugfixing means typing**

# HW Interfaces

- Don't change keyboards/layouts to write faster
- Do it to **avoid strain** on your fingers and wrists

10: Pain leads to shortcuts

20: Shortcuts lead to bugs

30: Bugfixing means typing

40: **goto** 10

# #define Protips

# Dependencies

# Dependencies

- Have a cost associated with using them

# Dependencies

- Have a cost associated with using them
  - License management

# Dependencies

- Have a cost associated with using them
  - License management
  - Repository and / or package size

# Dependencies

- Have a cost associated with using them
  - License management
  - Repository and / or package size
  - Transitive dependencies

# Dependencies

- Have a cost associated with using them
  - License management
  - Repository and / or package size
  - Transitive dependencies
    - Version conflicts

# Dependencies

- Have a cost associated with using them
  - License management
  - Repository and / or package size
  - Transitive dependencies
    - Version conflicts
- You cannot abstract away understanding

# Dependencies

- Have a cost associated with using them
  - License management
  - Repository and / or package size
  - Transitive dependencies
    - Version conflicts
- You cannot abstract away understanding
  - only delegate responsibility**

# Planned interfaces

# Planned interfaces

- Design and document your public API

# Planned interfaces

- Design and document your public API
- Don't make things **API by accident**

# Planned interfaces

- Design and document your public API
- Don't make things **API by accident**
- Public API needs to have proper documentation

# Planned interfaces

- Design and document your public API
- Don't make things **API by accident**
- Public API needs to have proper documentation
- Good API design takes a lot of effort

# Planned interfaces

- Design and document your public API
- Don't make things **API by accident**
- Public API needs to have proper documentation
- Good API design takes a lot of effort
  - **But is worth it!**

# Avoid defaults in code

# Avoid defaults in code

- Put defaults in external configuration

# Avoid defaults in code

- Put defaults in external configuration
  - Possible to change without recompiling

# Avoid defaults in code

- Put defaults in external configuration
  - Possible to change without recompiling
  - Makes structure of the code cleaner

# Avoid defaults in code

- Put defaults in external configuration
  - Possible to change without recompiling
  - Makes structure of the code cleaner
  - Avoids scattering same "default" in multiple places (DRY)

# Avoid shared mutable state

# Avoid shared mutable state

- Consequences of shared mutable state:

# Avoid shared mutable state

- Consequences of shared mutable state:
  - Interactions are hard to calculate

# Avoid shared mutable state

- Consequences of shared mutable state:
  - Interactions are hard to calculate
  - Almost always assumes single-threaded execution

# Avoid shared mutable state

- Consequences of shared mutable state:
  - Interactions are hard to calculate
  - Almost always assumes single-threaded execution
  - Hard to reason and verify impact of changes

# All the possibilities!

# All the possibilities!

```
public int mutable() {  
    int x = 1;  
    int y = 2;  
    int z = 3;  
    ...  
    return x + y + z;  
}
```

# All the possibilities!

```
public int mutable() {  
    int x = 1;  
    int y = 2;  
    int z = 3;  
    ...  
    return x + y + z;  
}
```

```
public int immutable() {  
    final int x = 1;  
    final int y = 2;  
    final int z = 3;  
    ...  
    return x + y + z;  
}
```

# Immutable

# Immutable

- Things that do not change frees the mind from calculating the interactions

# Immutable

- Things that do not change frees the mind from calculating the interactions
- Use **const** / **final** whenever possible

# Immutable

- Things that do not change frees the mind from calculating the interactions
- Use **const** / **final** whenever possible
- Favor immutability in collections and data

# Immutable

- Things that do not change frees the mind from calculating the interactions
- Use **const** / **final** whenever possible
- Favor immutability in collections and data
  - Persistent collections can be very efficient

# Expressions > Statements

# Expressions > Statements

- Statements do not "**return**" anything

# Expressions > Statements

- ⦿ Statements do not "**return**" anything
  - ⦿ They can only produce results by writing to shared memory

# Expressions > Statements

- Statements do not "**return**" anything
  - They can only produce results by writing to shared memory
  - This leads to having a lot of variables

# Expressions > Statements

- Statements do not "**return**" anything
  - They can only produce results by writing to shared memory
  - This leads to having a lot of variables
  - Remember what we said about immutability

# Expressions > Statements

# Expressions > Statements

```
public int foo() {  
  int x;  
  if (smth) {  
    x = 5;  
  } else {  
    x = 10;  
  }  
  ...  
  return result;  
}
```

# Expressions > Statements

```
public int foo() {  
    int x;  
    if (smth) x = 5;  
    else x = 10;  
    ...  
    return result;  
}
```

# Expressions > Statements

```
public int foo() {  
    final int x =  
        smth ? 5 : 10;  
    ...  
    return result;  
}
```

# Versioning

# Versioning

- Very few **enjoy** designing versioning

# Versioning

- Very few **enjoy** designing versioning
- Very few **enjoy** evolving an unversioned system

# Versioning

- Very few **enjoy** designing versioning
- Very few **enjoy** evolving an unversioned system
- Guess which one of these usually lasts longer

# Bad comments

# Bad comments

- Comments also rot

# Bad comments

- Comments also rot
- Never commit commented out code.

If it isn't worth compiling, it's not  
worth **maintaining**.

# Don't Copy Paste

# Don't Copy Paste

- Copy-paste violates DRY

# Don't Copy Paste

- Copy-paste violates DRY
- Use Cut-paste

# Don't Copy Paste

- Copy-paste violates DRY
  - Use Cut-paste
  - If you Cut-paste-paste-\*

# Don't Copy Paste

- Copy-paste violates DRY
  - Use Cut-paste
  - If you Cut-paste-paste-\*
  - **Stop it**

# Failures will occur

# Failures will occur

- Fault tolerance impacts system design

# Failures will occur

- Fault tolerance impacts system design
  - Think about failures up front

# Failures will occur

- Fault tolerance impacts system design
  - Think about failures up front
  - Avoid mixing concerns in code

# Failures will occur

- Fault tolerance impacts system design
  - Think about failures up front
  - Avoid mixing concerns in code
- Overload is a type of failure

# Failures will occur

- Fault tolerance impacts system design
  - Think about failures up front
  - Avoid mixing concerns in code
- Overload is a type of failure
  - How should the system behave?

# Pride in work

# $\Sigma$ -ary

# $\Sigma$ -ary

- Creating successful "bit rot" resistant software boils down to:

# $\Sigma$ -ary

- Creating successful "bit rot" resistant software boils down to:
  - Culture

# $\Sigma$ -ary

- Creating successful "bit rot" resistant software boils down to:
  - Culture
  - Process

# $\Sigma$ -ary

- Creating successful "bit rot" resistant software boils down to:
  - Culture
  - Process
  - Practices

# $\Sigma$ -ary

- Creating successful "bit rot" resistant software boils down to:
  - Culture
  - Process
  - Practices
- And last but not least:

# $\Sigma$ -ary

- Creating successful "bit rot" resistant software boils down to:
  - Culture
  - Process
  - Practices
- And last but not least:
  - **Continuously improving them**

Time is the fire  
in which we burn  
– Delmore Schwarz

EOF